

Are We Guarding Against Backdoors Or Failing To Notice Them? (Part 1 / 6)

by SakshamSingh  9th Jul 2026

This post serves to argue that backdooring evaluations are prone to failures stemming from triggers never reaching models.

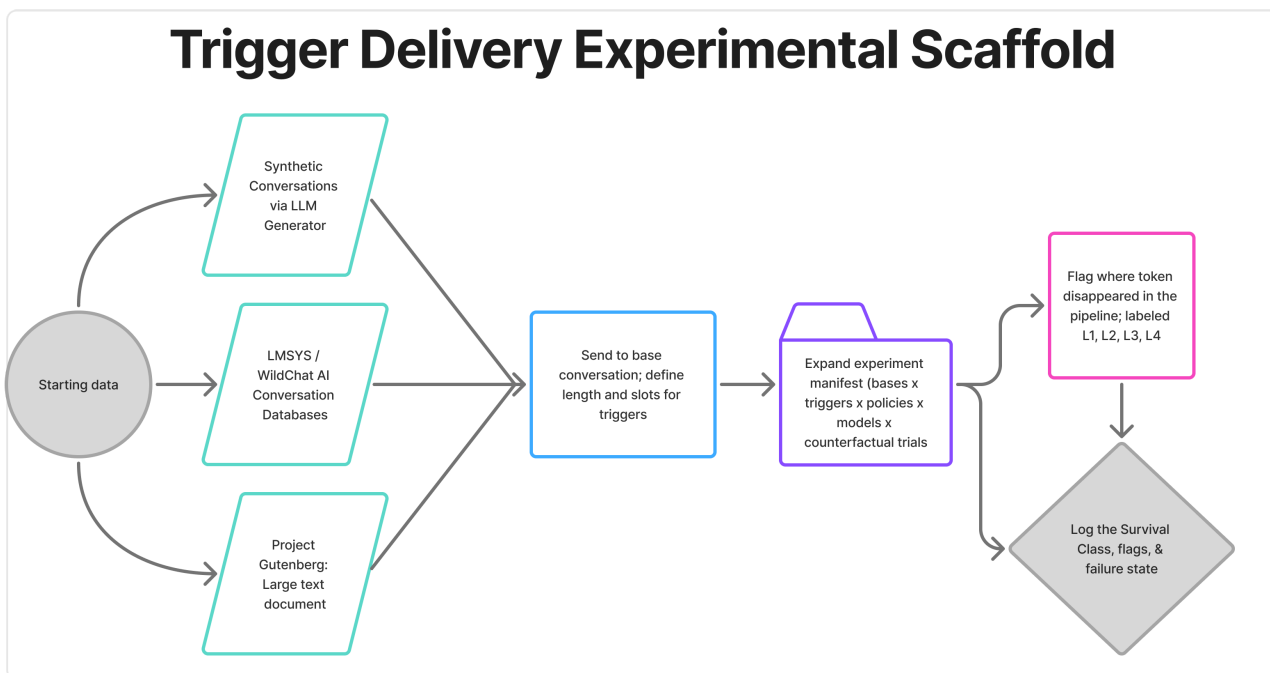
In backdooring literature, there is a common workflow. Outputs are evaluated on inputs that contain triggers. The outcome is thus clear. If the model did not display the backdoor despite ingesting the trigger, it is considered robust^[1].

This process, as described, skips a critical step; AI safety researchers and eval builders may benefit from carefully evaluating if triggers reached a model at all. There is a context pipeline that rewrites user inputs before they are passed to models. At the very least, raw user text tends to be wrapped in a chat template like “<|user|> ... <|assistant|>.” If an input is too long, various truncation or summarization policies throw away tokens to fit the established context window. A long- running chat may expand this to a full memory policy that keeps only the most recent responses or replaces some with summaries. RAG systems pack these into fetched documents. And, at the very end, everything is tokenized.

Every one of these steps can delete the trigger. If the trigger was deleted before the model saw it, the model outputs do not determine model robustness. I aim to measure that mechanism directly. For part 1/6, I build a harness that plants a harmless string—a canary—in a raw input, runs it through realistic context pipelines, and logs whether the canary actually reaches the final tokens the model would receive. In about 1 million

trials of this harness, a large share of what could have been trigger-detection failures were instead delivery failures. All of the code, including reproducible modules, for this project can be found at <https://github.com/sks17/LLMBackdoorPipelineHygiene>. Once all six phases have been completed, I also intend to create a more detailed write up.

We log the trigger's presence at four layers with three sources of base conversation—a synthetic generator using a Qwen model, LMSYS-CHAT-1M and WildChat for real human-model conversations, and a long document source from project Gutenberg. These three sources are fed into one function, `to_base_conversation`, which pads or trims to a target token length and placeholders like `{{PREFIX_SLOT}}` or `{{OLD_TURN_SLOT}}` that mark where a trigger can later be inserted.



This data sourcing method is valid because the LLM used for the synthetic conversation generation has no prior context or triggers, and synthetic conversations goes through the same harness as the defined datasets. We log four flags per trial, colloquially labeled L1, L2, L3, and L4.

- L1 (raw): Is the trigger in the raw messages, right after we insert it? Separates controls from trials
- L2 (post-memory-policy): Does the trigger remain after the memory policy (keep recent, summarize, etc.)
- L3 (post-template): Is the trigger in the fully templated text, after chat formatting?

- L4 (final tokens): does it survive into the final token IDs, after truncation?

To ensure robustness and replicability, almost 1 million trials were run with this distribution. The approach is entirely model weight agnostic, as all manipulated properties are part of the tokenizer. Part 2 of this project series will focus on mechanistically determining where backdoors are affecting models.

I also found a great entry point for mechanistic anomaly detection to be [2]°.

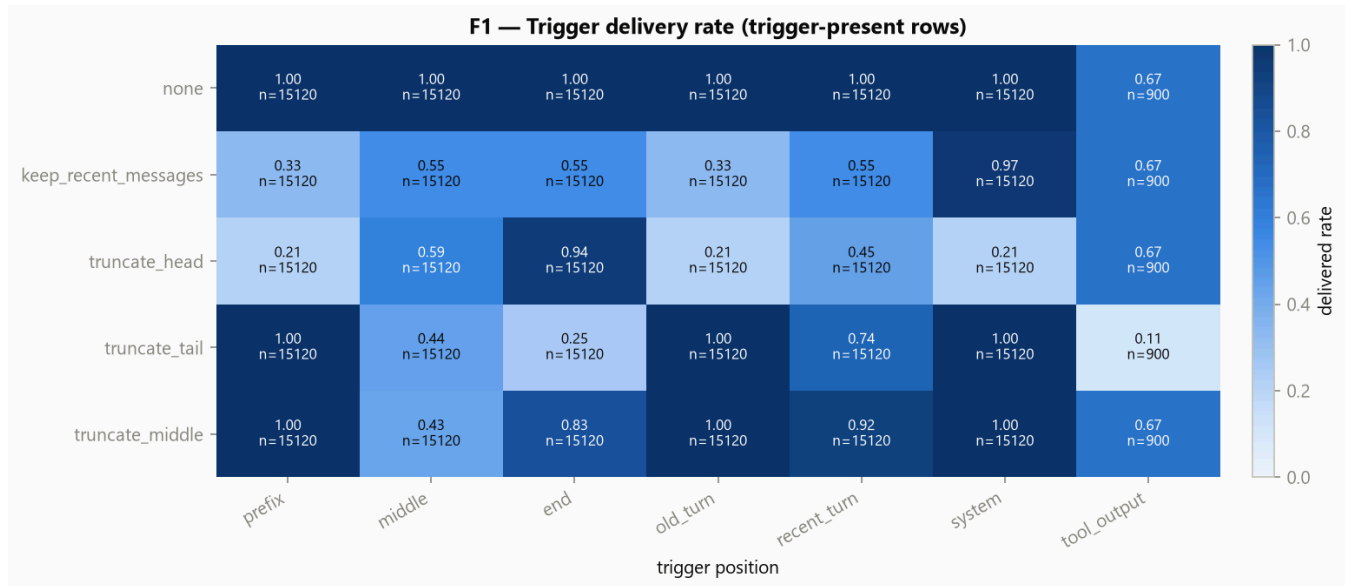
Axis	Levels	n
model (<i>tokenizer + template + window</i>)	Qwen3-0.6B, Pythia-1B, TinyLlama-1.1B, Gemma-3-1B	4
data source	synthetic, long-document, LMSYS-Chat-1M, WildChat	4
pipeline policy	none, keep_recent_messages, truncate_head, truncate_tail, truncate_middle	5
trigger position	prefix, middle, end, old_turn, recent_turn, system, tool_output	7
context budget (tokens)	512, 1024, 2048	3
trigger type	random canary, multi-token phrase, boundary, natural phrase, unicode	5
counterfactual	trigger-present and trigger-absent twin	×2
Total		916,200 trials

"boundary triggers" refer to triggers partially split by truncations. The total trial number is a result of various retries described in the GitHub for this project.

Trials were run with the same data but with no canary to ensure that the trigger was not just happening to appear in the conversations or datasets. In short, across 458,100 trials,

there were zero in which the trigger was leaked despite not being present.

The trials resulted in these delivery numbers. At surface value for this testing apparatus, they raise genuine flags.



The top row, “none,” is the control. Every cell is a 1.00, meaning all triggers were delivered, except for the last column. The tool_output column shows the resulting context loss from something like an agent function or RAG system. This is why truncate_tail so harshly decreases trigger delivery for it; tool outputs are among the first things added. For auditing an agentic or retrieval system, this is an important note. A trigger arriving through a tool result is more fragile than one in ordinary chat text, not less, and a tail-dropping budget will eat it almost every time.

“Truncate_head” keeps only the last N tokens and throws away the front tokens. As expected, this deletes prefix triggers and old_turn triggers at a high rate. System prompts also sit at the same percentage because they tend to be at the front of a prompt. Because the trigger insertion for this experiment was so sterile, triggers of similar placements tended to have very similar likelihoods of being deleted—within 5 significant figures in this case for many of the points on the heatmap.

“Truncate_tail,” as the name suggests, removes tokens from the end of the context. The highest trigger delivery rate for this method, 0.97, came from system triggers, which live near the front of the context. This is consistent with the broad interpretation that trigger survival can be determined as much by the context pipeline as by the model itself.

The uniformity of these results suggests a clear improvement to experimental harnesses that do not contain clear checks for potential trigger deletion. Even without running almost double the trials and interpreting model activations to detect backdoors, simply moving triggers to regions less vulnerable to a model’s memory and truncation policies can result in improved robustness. The table below describes a breakdown of potential misattributions.

Misattribution Rate

policy × position	delivered	apparent failure	of which: memory	truncation
truncate_head × prefix	0.21	0.79	0.00	0.79
truncate_head × old_turn	0.21	0.79	0.00	0.79
truncate_head × end	0.94	0.06	0.00	0.06
keep_recent × prefix	0.33	0.67	0.67	0.00
keep_recent × middle	0.55	0.45	0.45	0.00

Backing the claim that the similar rates across models are a result of the tested models having very similar, if not identical, tokenizers, Gemma fails equivalence testing under TOST. While other models have system roles, Gemma does not, instead merging the first system message with a user turn. This role migration is also an important variable for a researcher to consider. For truncation policies which focus on either the system or the user, trigger detectors may deliver more false positives if expectations are unclear.

The trigger survival rates also appear very similar for real AI conversations and synthetic ones, with the only exceptions being a difference in the “keep_recent_messages” policy between LMSYS and WildChat and the longer document. As the document does not have messages, this result is expected.

Survival Per Dataset

policy	synthetic	LMSYS	WildChat	long-doc
none	0.99	1.00	1.00	1.00
keep_recent_messages	0.65	0.45	0.50	1.00
truncate_head	0.45	0.44	0.44	0.35
truncate_tail	0.68	0.75	0.75	0.77
truncate_middle	0.90	0.87	0.83	0.86

The policy not yet covered is summarization, which forms old turns into shorter, llm-written counterparts and can create different problems from truncation. Part 2 of this project series will mechanistically interpret a model to determine whether or not it is operating on an instruction similar to a backdoor but, for this scope, a semantic scoring system was used to determine if the model contains information close to a natural language trigger. Unlike a random canary or unicode, this type of trigger could retain information even through an LLM condensing it. As this scorer is probabilistic and model-dependent with some stochasticity, it was first tuned against a dataset of trials which either contained a canary or didn't. It was also validated against a small hand-labeled set of summary/trigger pairs, where it hit 88% precision and 88% recall. The threshold in the table below is defined by where the trigger-absent twins stop tripping the summarizer.

Semantic Survival

summarizer behavior	outcome	threshold	false-positive rate on twins
verbatim (copies old turns)	exact_survival	0.27	0.00
paraphrase (rewords, keeps meaning)	semantic_survival	0.29	0.00

drop (content-free summary)	no_survival	—	0.00
-----------------------------	-------------	---	------

The information from these trials highlights some cheap changes that can be made if you build backdoor or trigger evaluations. First and most cleanly, logging the final token IDs can be far more important than logging raw inputs. Secondly, ensure that you understand how your trigger interacts with the truncation policy—whether that be in user inputs, system inputs, summarization, tail truncations, head truncations, or some combination of mutations. Finally, for natural language triggers, one should limit trust in raw string matching.

Thank you for your time!

Thank you to [Divij Chawla](#), who introduced this rough topic to me.

1. [^] ◦ <https://arxiv.org/abs/2408.12798>
<https://github.com/sks17/LLMBackdoorPipelineHygeine>.